Amortized Analysis

An Amortized analysis is an strategy for analyzing a sequence of operations to show that the **average cost per operation is small**, even though a single operation within the sequence might be expensive

- It refers to finding the average running time per operation over a worst case sequence of operations

- It is technique for analysing algorithms ***under some conditions***

- Amortized analysis differs from average case performance in that probability is not involved;

  In average case analysis, we are averaging over all possible inputs

- Amortized analysis guarantees the time per operation over worst case performance

- Amortized analysis assumes worst case input and typically does not allow random choices

Techniques used in Amortized Analysis

- ***Aggregate Analysis*** determines the upper bound T(n) on the total cost of a sequence of n operations, then calculates the average cost to be T(n)/n

- ***Accounting Method*** determines the individual cost of each operation, combining its intermediate execution time and its influence on the running time of future operations

- ***Potential Method*** like an accounting method, but overcharges operations early to compensate for undercharges later

Amortized bounds
- assign amortized cost for each operations such that it "preserve the sum"

∑ amortized cost >= ∑ Actual cost

- Idea is that the worst case operation can alter the state in such way that worst case cannot occur again for a long time, thus “amortizing” its cost

- It is applicable to some special set of problems like hash tables, disjoint set, splay trees etc.

- It has significance where first set of input affects the next set of inputs

Case 1: Find kth smallest element

```
int find_Kth _smallest ( int A[], int n, int k)
{
If (! Sorted)
Sort(A,n)
return A[k]
}
```

Running Time is O(n log n)

- ***To find n different kth smallest element***

K= 1st, 10th, 50th ….

Worst Case n* (n log n)

Amortized cost per operation

= Total cost of k operations/k

Amortized Analysis

(nlogn)+O(1)+O(1)+…….

nlogn +1+1+…..(n–1) = (nlogn+n)/n

= O(logn+1) =logn

Scenario - doubling the space

Ex: (Adding puppies)

- If there is a space
  - –Add puppy in free space O(1)

If there is no space left

- -buy a space with 2 X storage (1 unit)
- -Move all puppies in new facility (N + 1 unit)
- -Total Cost = 1+N+1 = O(N)

Add Puppy	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
N puppies in facility	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Spaces available in current facility	1	1	2	4	4	8	8	8	8	16	16	16	16	16	16	16
Time to buy new facility	0	1	1	0	1	0	0	0	1	0	0	0	0	0	0	0
Time to move puppies from old place to new	0	1	2	0	3	0	0	0	7	0	0	0	0	0	0	0
Time to move new puppy into empty space	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Total Time	1	3	4	1	6	1	1	1	10	1	1	1	1	1	1	1
Total Time in Segment	1	3	5		10				17							
Number of add puppy operations in segment	1	1	2		4				8							
Average Time in segment	1	3	2.5		2.5				2.125							

Add Puppy	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
N puppies in facility	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Spaces available in current facility	16	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32
Time to buy new facility	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Time to move puppies from old place to new	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Time to move new puppy into empty space	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Total Time	18	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Total Time in Segment	34																
Number of add puppy operations in segment	16																
Average Time in segment	2.125																

Add Puppy	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
N puppies in facility	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Spaces available in current facility	1	1	2	4	4	8	8	8	8	16	16	16	16	16	16	16
Time to buy new facility	0	1	1	0	1	0	0	0	1	0	0	0	0	0	0	0
Time to move puppies from old place to new	0	1	2	0	3	0	0	0	7	0	0	0	0	0	0	0
Time to move new puppy into empty space	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Total Time	1	3	4	1	6	1	1	1	10	1	1	1	1	1	1	1
Total Time in Segment	1	3	5		10				17							
Number of add puppy operations in segment	1	1	2		4				8							
Average Time in segment	1	3	2.5		2.5				2.125							

Add Puppy	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
N puppies in facility	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Spaces available in current facility	16	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32	32
Time to buy new facility	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Time to move puppies from old place to new	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Time to move new puppy into empty space	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Total Time	18	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Total Time in Segment	34																
Number of add puppy operations in segment	16																
Average Time in segment	2.125																

Observation

- –it is only expensive when full
- Many cheap add after expensive one
- Spread out the expensive cost uniformly

Sequence of Operations PUSH, POP, MULTIPOP

Actual Cost

- PUSH 0(1) 1
- POP 0(1) 1
- Multipop (S,k) min(S,k)

Amortized analysis

- PUSH 2
- POP 0
- Multipop (S,k) 0

```
Multipop(S,k)
While not stackEmpty(S) and k<>0
Do POP S
K=k-1
```

Total cost for n sequence of operations = 2n

Amortized cost/operation

= Total

Cost / Total Ops

= 2n / n

= 2

(constant)

Therefore Effective cost per operation in constant i.e. O(1).